

MemoryGit: AIの長期記憶を安全に使うための状態制御型記憶参照システム

Status: public-facing preprint draft Date: 2026-05-15 Code release: not included Primary audience: Japanese technical readers

要旨

長期記憶を持つAIシステムでは、関連する情報を検索できるだけでは不十分である。記憶は、現在有効なもの、過去の履歴として残るもの、削除済みのも、確認待ちのもの、衝突しているもの、特定の project や session にだけ有効なもの、そもそも記憶として読んではいけないものに分かれる。

本稿では、AIの長期記憶を単なる検索問題ではなく、「どの記憶を、どの状態として、どの時点で、どの文脈なら使ってよいか」を判断する制御問題として扱う。この考え方を、状態制御型記憶参照と呼ぶ。

MemoryGit は、長期記憶を単なる検索対象ではなく、状態・時点・権限・文脈に基づいて参照を制御するための状態制御型記憶参照システムである。MemoryGit は、記憶の保存、昇格、削除、scope、監査、参照時点、使用許可を分けて扱う。特に参照時には、イベント権限、非参照文脈の抑制、記憶状態の読み分け、近接状態の優先順位、候補状態の制限、その時点の snapshot、最終的な使用判断を分離する。

内部実験では、MemoryGit の store、snapshot、値の取得自体よりも、参照前段の「これは記憶として読んでよい文脈か」「current / 過去 / 削除済み / 要確認 / 衝突のどれか」を読む部分が主要な失敗源であることが繰り返し確認された。最新の大きめの long-log 評価では、v5aq と v5ar の2つの別 140 行評価面で、5つの family すべてが selected / route / no-route = 1.000、false route = 0、value miss = 0、missing reference = 0、leakage = 0 を達成した。さらに v5bb では、fresh localhost MemoryGit API replay から export envelope、request/key split、scope guard、selected-readout evaluation までを一つの決定的 product-path chain として実行し、280 request rows で selected / route / no-route = 1.000、false route = 0、value miss = 0 を得た。

ただし、本稿は任意自然会話、実プロダクト traffic、複数モデル一般化、一般的な継続学習、完全自律AGI 自己改善を解いたとは主張しない。本稿の主張は、テストされた MemoryGit 形式の長期ログ契約において、状態制御型記憶参照が長期記憶の安全な使用に有効である、という限定されたものである。

1. 背景

LLM を使ったAIシステムは、単発の応答生成器から、継続的な状態を持つエージェントへ移りつつある。ユーザーの好み、プロジェクトの文脈、過去の依頼、削除指示、共有禁止、関係性、タスク状態などを保持するシステムが増えている。

しかし、記憶を持つことは、その記憶を常に使ってよいことを意味しない。

ある記憶は今も有効かもしれない。別の記憶は過去の履歴かもしれない。ある記憶は削除済みかもしれない。ある記憶は特定の project だけで使ってよいものかもしれない。ある発言は本人の事実ではなく、仮定、冗談、引用、他AIの出力、創作、教材、監査ログかもしれない。

したがって、長期記憶AIの問題は、単に「似ている記憶を探すこと」ではない。

中心問題は次である。

保存された記憶を、今、どの資格で使ってよいか。

2. RAG と要約メモリの限界

RAG は有用である。文書や会話を chunk にし、embedding を作り、query に近い chunk を取り出す。この仕組みは、知識検索や文書参照では強い。

しかし、長期パートナーAIの記憶では、RAG だけでは危ない。

混同されるもの	起きる事故
発話事実と内容事実	「仮に大阪在住として」が住所記憶になる
本心とテスト	「私は火星出身です。覚える？」が属性になる
一時感情と長期意思	「会社辞めたい」が退職予定になる
引用と本人事実	他AIの「私は東京在住」がユーザー住所になる
創作と現実	小説設定が本人プロフィールになる
project / session / off	その場限りの仮定が global memory になる
no-store / 削除	類似検索で再び出てくる
permission	「共有しないで」が通常の実事メモになる
conflict	社会的・構造的対立が単一 fact に潰れる

要約メモリにも別の危険がある。要約は読みやすいが、曖昧さを消しすぎることもある。

たとえば、元の発話が仮定、引用、冗談、一時感情、撤回、共有禁止だったとしても、要約では次のような自然な事実文に変わることがある。

ユーザーは大阪に住んでいる。
 ユーザーは転職を考えている。
 ユーザーは家族の病歴を共有した。

これは長期記憶では危険である。要約は情報圧縮であると同時に、責任境界を消すことがある。

MemoryGit は RAG を否定しない。RAG は候補取得として使える。しかし、候補取得は記憶統治ではない。候補を使ってよいかを決める状態制御が必要である。

3. MemoryGit の基本概念

MemoryGit は、記憶を chunk ではなく、状態を持つ claim として扱う。

少なくとも次の状態を区別する。

状態	意味
current	今も有効な記憶として使ってよい
archived	過去の履歴として残るが、現在値としては使わない
erased	削除済み、または回答に使ってはいけない
no-current	現在値が存在しない
review_required	人間確認が必要
conflict	候補が衝突している
no-route	そもそも記憶参照ではない

重要なのは、これらが単なるラベルではなく、実際のAIの振る舞いを変える制御状態である点である。

たとえば、ある情報が current ならAIは使ってよい。archived なら現在値として断定してはいけない。erased ならプロンプトにも回答にも戻してはいけない。review_required なら人間確認なしに使ってはいけない。no-route なら、そもそも記憶参照処理を起動してはいけない。

4. 状態制御型記憶参照

MemoryGit の参照制御は、概念的には次の層に分かれる。

イベント権限と scope の確認
 -> 記憶参照文脈かどうか
 -> 記憶状態の読み分け
 -> 近接状態の優先順位

- > 狭い既知誤読の補正
- > 候補状態からの選択
- > その時点の snapshot
- > 最終的な使用 / review / reject

各層の役割は次の通りである。

層	役割
イベント権限 / scope	そのイベントが記憶参照に使えるかを見る
非参照文脈の抑制	背景説明、引用、教材、監査ログを読まない
記憶状態の読み分け	current / 過去 / 削除済み / 要確認 / 衝突を読む
優先順位	近い状態が衝突したときに順序を決める
狭い補正	既知の誤読ポケットだけを直す
候補状態ゲート	MemoryGit 内の候補状態からだけ選ぶ
snapshot	参照イベント時点の値を読む
使用判断	出力する、review する、拒否する

一発の大きな分類器で全部を決めないのが重要である。

実験では、広い分類器や広い semantic augmentation は、一部の失敗を直しても別の失敗を生むことが多かった。たとえば、削除済み表現を強く拾うと current が erased に吸われる。review 表現を広く拾うと erased や no-current の境界を壊す。answer-exclusion を広く拾うと、背景説明の barred / blocked まで erased として扱う。

そのため、MemoryGit は「賢い一発分類」ではなく、層と優先順位と narrow gate の組み合わせとして設計している。

5. 実験で見えたこと

実験では、以下のパターンが繰り返し確認された。

1. MemoryGit store、候補状態、snapshot、値取得は多くの場合壊れていない。
2. 主な失敗は、参照前段の discourse / state-frame parser に集中する。
3. 背景説明、引用、教材、監査コメントに含まれる状態語が、操作指示として誤読されやすい。
4. 同一 holdout 上の修理と、別 holdout への転移は分けて評価する必要がある。
5. 広い修理より、狭い発火条件と優先順位を持つ修理の方が副作用が少ない。

評価台帳では、次を分けて記録した。

種類	意味
negative boundary	どこで壊れるか
post-repair	壊れた同じ評価面で修理できたか
fresh transfer	修理を凍結して別評価面でも通るか

この区別は重要である。同じ holdout 上で 1.000 になっても、それはまだ一般化ではない。別の評価面に転移して初めて、より強い証拠になる。

6. 現時点の代表結果

現時点で最も強い結果は、long-log 側の v5aq / v5ar と、live API product-path 側の v5bb である。

long-log 側では、v5ap の修理スタックを凍結したまま、別々の 140-row larger long-log scenario surface に適用した。v5aq に続き、v5ar でも以下の結果を得た。

指標	結果
valid generated contexts	140
families	5
quarantine	0
missing references	0
leakage	0
value misses	0
false routes	0
selected / route / no-route	1.000 / 1.000 / 1.000

これは、テストされた MemoryGit 形式の deterministic scenario-corpus-derived long-log contract ではかなり強い結果である。

live API product-path 側では、v5bb が次の一気通貫の決定的 chain を実行した。

fresh localhost MemoryGit API replay

- > export envelope
- > product-style request / private answer-key split
- > scope-guarded request surface
- > selected-readout evaluation

v5bb では、live source rows 7、export rows 28、request rows 280、answer-key rows 280、audit errors 0 であった。scope guard つき resolver は selected / route / no-route = 1.000、false route = 0、value miss = 0 を達成した。一方、text-only baseline は route accuracy 0.186、scope guard を外した比較は 0.721 に止まった。これは、長期記憶の参照には本文だけでなく、event envelope、scope matching、missing-field safe review が重要であることを示している。

ただし、これらは実プロダクト全 traffic の保証ではない。long-log 側は生成された評価契約、v5bb は決定的 localhost API replay における陽性結果である。

7. 自己改善ループへの接続

MemoryGit の失敗台帳は、品質ゲート駆動の自己改善管理にも接続できる。

流れは次のようになる。

失敗を観測する

- > 失敗を分類する
- > 改善案を作る
- > sandbox で試す
- > 品質ゲートを走らせる
- > 改善 / 悪化を判定する
- > review packet を作る
- > 採用 / 却下の理由を台帳に残す

ただし、これは完全自律AGIの自己改造ではない。現在の主張は、安全制約、sandbox、rollback、人間レビューを前提にした改善実験管理である。

禁止するものは明確にしている。

- 自動 merge
- 本番 repo への直接 patch
- 本番記憶変更
- safety gate 緩和
- eval threshold 緩和
- provider policy 緩和
- 全部 block / 全部 review / 全部 no-store による見かけ上の改善

この境界を守ること、自己改善を「危険な自動改造」ではなく、「品質ゲートつきの改善実験」として扱う。

8. まだ言わないこと

本稿は次を主張しない。

- 任意自然会話を完全に解いた
- 実プロダクト traffic で完全に安全
- どのLLMでも同じように動く
- free generation を解いた
- 一般的な継続学習を解いた
- AGIの完全自律自己改善が完成した

現時点の主張は、より限定的である。

テストされた MemoryGit 形式の長期ログ契約では、状態・文脈・時点・候補状態を分けて制御することで、長期記憶の参照を安定化できる。

また、決定的な localhost API replay では、export envelope、request/key split、scope guard を通した product-path handoff でも同じ参照制御が機能する。

9. 今後の検証

次に必要な検証は以下である。

検証	目的
さらに別の long-log / real-log-derived surface	v5aq / v5ar が一回限りでないかを見る
第三生成器の holdout	生成器依存を下げる
sanitized real-log-derived holdout	生成文から実ログ風へ寄せる
cross-model comparison	モデル依存を測る
layer ablation	どの層が本当に効いているかを見る
live API replay の拡張	v5bb の決定的 product-path chain をより多い scenario に広げる
provider-path evaluation	モデルに渡る prompt / 応答で漏れないかを見る

特に重要なのは、生成された評価面から、匿名化された実ログ風評価面へ移ることである。

10. まとめ

AIの長期記憶で大事なものは、たくさん覚えることだけではない。

重要なのは、

覚えた情報を、今、どの状態で、どの時点で、どの文脈なら使ってよいかを制御することである。

RAG は似ている情報を探す。

MemoryGit は、その情報を今使ってよいかを判断する。

この差分が、長期記憶AIでは本体になる。

中心命題は次である。

長期記憶AIに必要なのは、記憶を多く保存することではなく、保存された記憶をどの状態・どの時点・どの文脈で使ってよいかを制御することである。

公開範囲について

本稿では、評価プロトコル、失敗分類、層構造、集計結果、主張境界を説明する。

一方で、以下は公開しない。

- 実装コード
- private packet
- answer key
- exact cue / override rule
- 内部運用メモ
- 未公開の製品安全評価詳細

理由は、製品安全性、評価ハック防止、未公開ノウハウ保護のためである。